

Lecture 13

Data Types

Contents of the lecture

Data Types

(1) Net data type

(2) Register data type

(a) “reg” data type

(b) “integer” data type

(c) “real” data type

(d) “time” data type

(3) Vectors



Dr. Yogesh Mishra
Professor of ECE< GMRIT

A variable in Verilog can be of two types:

(A) Net

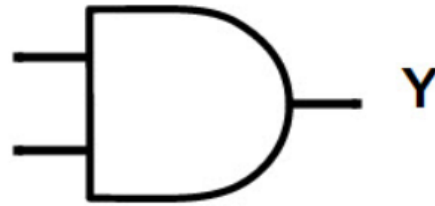
- (i) Must be continuously driven
- (ii) Cannot be used to store a value.

(B) Register

- (i) Retains the last value assigned to it.

(A)Net Data Type

- (i) Nets represents connection between hardware elements.
- (ii) Nets are continuously driven by the outputs of the devices they are connected to



Net “Y” is continuously driven by the output of the AND gate.

- (iii) Nets are 1-bit values by default unless they are declared explicitly as vectors.

Default value of a net is “z”

- (iv) Various net types:

wire, wand, wor, tri, supply0, supply1

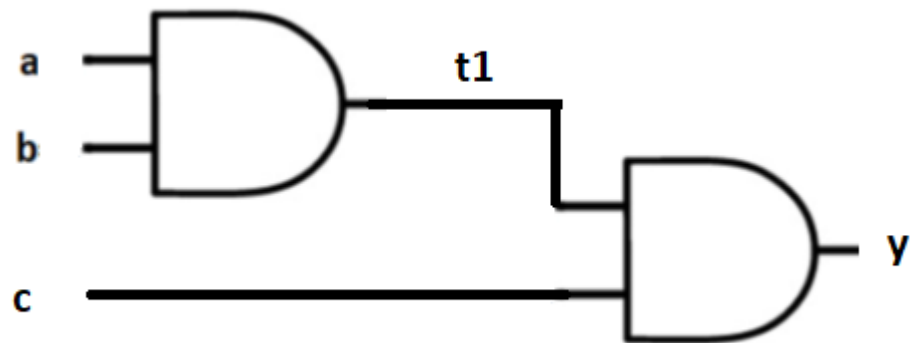
- (v) “wire” and “tri” are same; when there are multiple drivers driving them the output is shorted.

- (vi) “wand” and “wor” insert “and” and “or” gated respectively at the junction.

- (vii) “supply0” and “supply1” model power supply

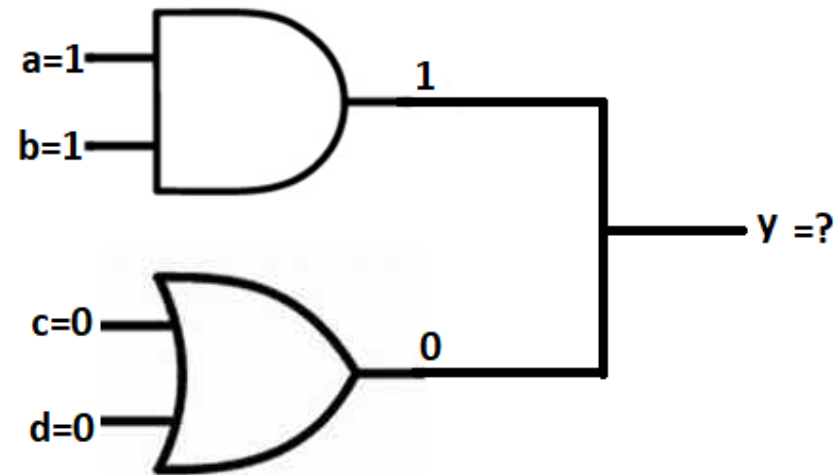
Example:

```
module multilevel_circuit(a,b,c,y);
input a,b,c;
output y;
wire t1; // net t1 is declared as wire
assign t1=a & b;
assign y=t1 & c;
endmodule;
```



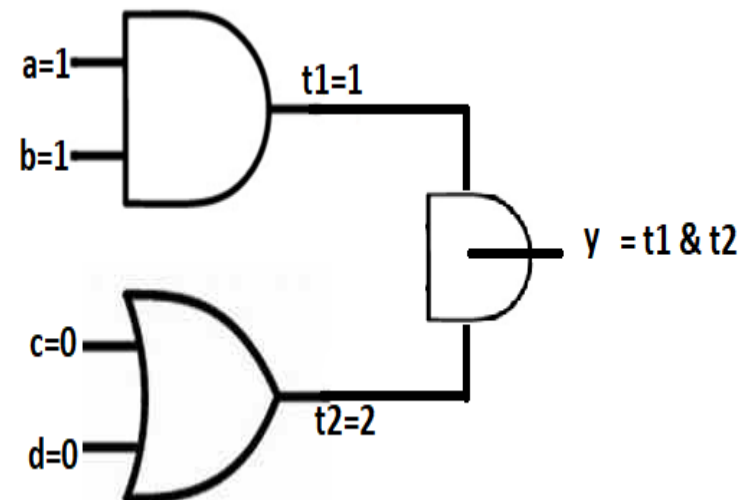
Example:

```
module use_of_wire(a,b,c,d,y);
input a,b,c,d;
output y;
wire y; // net y is declared as wire
assign y=a & b;
assign y=c | d;
endmodule
```



Example:

```
module use_of_wand(a,b,c,d,y);
input a,b,c,d;
output y;
wand y; // net y is declared as wand
assign y=a & b;
assign y=c | d;
endmodule
```



Lecture 14

Data Types (Continue)

Contents of the lecture

Data Types

(1) Net data type

(2) Register data type

(a) “reg” data type

(b) “integer” data type

(c) “real” data type

(d) “time” data type

(3) Vectors



Dr. Yogesh Mishra
Professor of ECE< GMRIT

A variable in Verilog can be of two types:

(A) Net

- (i) Must be continuously driven
- (ii) Cannot be used to store a value.

(B) Register

- (i) Retains the last value assigned to it.

(B) Register Data Type

- (i) In Verilog, a “register” is a variable that can hold a value (whereas “net” is continuously driven and cannot hold any value).
- (ii) “register” does not mean that a hardware register will be mapped during synthesis.
- (iii) Combinational circuits can also use register type variable.

Following “register” data types supported by Verilog:

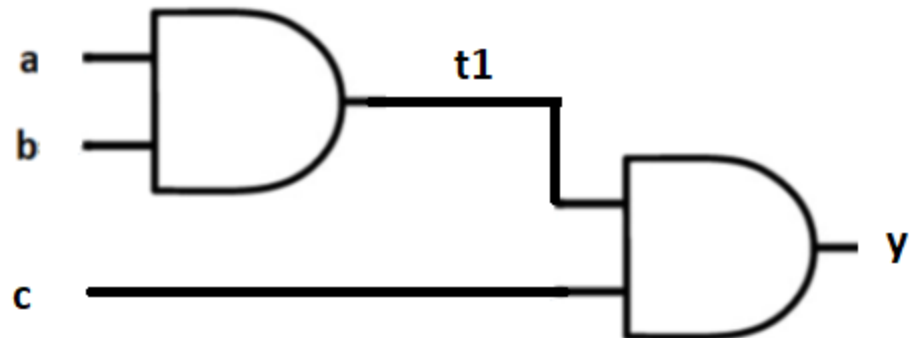
- (a) reg – Mostly used
- (b) integer
- (c) real
- (d) time

(a) “reg” Data Type

- Default value of a “reg” data type is “x”
- Default declaration of “reg” specifies 1-bit

Example:

```
module multilevel_circuit(a,b,c,y);  
input a,b,c;  
output y;  
reg y,t1; // t1 and y are declared as “reg” Register type  
always @(a,b,c)  
begin  
t1=a & b;  
y=t1 & c;  
end  
endmodule;
```



(b) “integer” Data Type

- It is used to manipulate quantities
- It is more convenient to use in situation like loop counting than “reg”
- Its default size is 32 bits

Example:

```
wire [15:0]a, b  
integer c;
```

```
c = a + b;
```

Size of c will be 17 bits (16 bits plus one carry)

(c) “real” Data Type

- It is used to store floating-point numbers
- When a real value is assigned to an integer, the real number is rounded off to the nearest integer.

Example:

real b;

integer c;

begin

b = 3.718;

c = b; // c will become 4

(d) “time” Data Type

- In Verilog, simulation is carried out wrt a logical clock called simulation time.
- The “time” data type is used to store simulation time.
- The system function “\$time” gives current simulation time.

Example:

```
time current_time;
```

```
initial
```

```
begin
```

```
....
```

```
current_time = $time;
```

- “net” or “reg” type variables can be declared as vectors of multiple bit widths (if the bit width is not specified, default size is 1-bit).
- Vectors are declared by specifying a range [range 1 : range 2], range 1 is always the MSB and range 2 is the LSB.

Example:

reg [31:0]a; // MSB is a[31], LSB is a[0]

reg [1:10]b; // MSB is b[1], LSB is b[10]

- Parts of a vector can be addressed and used in an expression.

Example:

```
reg [7:0] x;  
reg [3:0] y;  
reg [3:0] z;
```

```
y = x[7:4];  
z = x[3:0];
```